

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к индивидуальному домашнему заданию
по дисциплине «Введение в нереляционные системы управления базами
данных»
Тема: Система антиплагиата для отчетов

Студенты гр. 3341

Ягудин Д.Р.
Лодыгин И.А.
Рябов М.Л.

Преподаватель

_____ Заславский М.М.

Санкт-Петербург
2026

ЗАДАНИЕ

Студенты Ягудин Д.Р.

Лодыгин И.А

Рябов М.Л.

Группа 3341

Тема проекта: система антиплагиата для отчетов

Исходные данные:

Создать веб-приложение, в которое можно загружать отчеты и получать метрики(процент оригинальности, индекс флеша и тд.)

Содержание пояснительной записки:

Содержание

Введение

Качественные требования к решению

Сценарий использования

Модель данных

Разработка приложения

Вывод

Приложение

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания:

Дата сдачи реферата:

Дата защиты реферата:

Студенты

Преподаватель

Ягудин Д.Р.
Лодыгин И.А.
Рябов М.Л.
Заславский М.М.

АННОТАЦИЯ

В рамках курсовой работы разработано веб-приложение командой по теме «Система проверки учебных отчётов на плагиат» с использованием графовой нереляционной СУБД Neo4j.

Приложение реализует интерфейсы и сценарии, описанные в техническом задании и сценариях использования: страницы входа и авторизации, Лента отчётов, Пошаговый процесс создания отчёта, Профиль студента, Детальный просмотр отчёта, Проверка на плагиат, визуализация графа связей, Статистика и аналитика, Поиск отчётов, CRUD-операции для студентов и отчётов, Экспорт/импорт всей базы данных в формате JSON.

СОДЕРЖАНИЕ

	Введение	6
1.	Постановка задачи	8
1.1.	Предлагаемое решение	8
1.2.	Качественные требования к решению	8
2.	Сценарии использования	10
3.	Модель данных	13
4.	Разработанное приложение	33
4.1	Краткое описание	33
4.2	Используемые технологии	33
5.	Заключение	35
	Список использованных источников	36
	Приложение А. Документация по сборке	0

ВВЕДЕНИЕ

Данные студенческих отчётов естественно принимают графовую форму: отчёты, студенты, структурные разделы (части) и текстовые фрагменты, а также их взаимосвязи (кто сдал отчёт, из каких разделов он состоит, какие фрагменты текста содержатся в каждом разделе, какие фрагменты являются заимствованными из других отчётов) моделируются как узлы (Report, Student, Part, Chunk) и рёбра (SUBMITTED, HAS_PART, CONTAINS).

Реляционные СУБД требуют множества таблиц и сложных JOIN-операций для представления и обхода таких связей, что снижает производительность при глубоком или широком обходе (например, при поиске всех отчётов, содержащих определённый фрагмент текста, или при построении цепочки заимствований через общие фрагменты) и усложняет формулировку запросов.

Графовые базы данных хранят связи как объекты первого класса, что позволяет выполнять переходы по графу с постоянной стоимостью на шаг независимо от общего объёма данных. Neo4j — одна из наиболее зрелых и широко используемых графовых СУБД — предоставляет декларативный язык Cypher для описания паттернов в графе. Запросы на Cypher для поиска отчётов с пересекающимися фрагментами, вычисления оригинальности (отношения уникальных фрагментов к общему числу) или выявления наиболее часто заимствуемых источников обычно короче и читабельнее эквивалентов на SQL с рекурсивными CTE в несколько раз.

Разработанное приложение «Антиплагиат» содержит десятки student-отчётов, сотни текстовых фрагментов (chunks) и множество перекрёстных связей между отчётами через общие фрагменты, что делает его практическим примером для демонстрации преимуществ графовых СУБД при работе со связанными текстовыми данными и задачами обнаружения заимствований.

Цель работы — разработка веб-приложения для хранения, анализа и визуализации связей между студенческими отчётами с использованием графовой СУБД Neo4j для выявления плагиата.

Приложение обеспечивает полный цикл работы с данными: импорт/экспорт в машиночитаемых форматах (JSON), загрузка и автоматический парсинг документов .docx с выделением структурных разделов (введение, основная часть, заключение, список литературы) и текстовых фрагментов, ручное добавление и редактирование отчётов и студентов, поиск и фильтрацию по названию, автору, группе, предмету, дате, индексу Флеша и оригинальности, а также полнотекстовый поиск по содержимому отчётов, интерактивную визуализацию связей в виде графа (отчёт → разделы → фрагменты с выделением заимствованных фрагментов) и формирование статистики по группам, предметам и авторам.

Выбранный стек технологий: Neo4j Community Edition в качестве графовой СУБД, Python с FastAPI для REST API, Jinja2 + Bootstrap для фронтенда (серверный рендеринг). Все компоненты контейнеризированы через Docker и оркестрированы с помощью Docker Compose для упрощённого развёртывания и воспроизводимости среды.

В пояснительной записке описаны постановка задачи, макет и сценарии использования, модель данных (графовая vs реляционная) с их сравнительным анализом, архитектура и применённые технологии, а также выводы по результатам реализации.

1. Постановка задачи

Необходимо разработать веб-приложение для хранения, анализа и визуализации связей между студенческими отчётами с использованием графовой СУБД Neo4j для выявления заимствований и проверки оригинальности текстов.

1.1. Предлагаемое решение

Предлагаемое решение — трёхзвенное веб-приложение, состоящее из: Neo4j 5.18 Community Edition в качестве графовой БД для хранения сущностей (отчёты, студенты, разделы, текстовые фрагменты) и связей между ними в виде узлов и рёбер

Серверная часть (backend) на Python 3.12 с FastAPI, предоставляющая REST API для всех операций над данными

Клиентская часть (frontend) — серверный рендеринг на Jinja2 + Bootstrap 5 + JavaScript, реализующий интерфейсы и сценарии использования

1.2. Качественные требования к решению

Удобство использования достигается за счёт интуитивного интерфейса с навигацией по основным разделам («Лента отчётов», «Студенты», «Поиск», «Статистика»), а также интерактивной визуализации графа связей с поддержкой масштабирования и перетаскивания узлов. Модальные окна и встроенные формы позволяют быстро просматривать и редактировать информацию без лишних переходов между страницами.

Контейнеризация реализована через Docker Compose, что позволяет запустить все компоненты системы (Neo4j, backend, frontend) одной командой без ручной установки зависимостей и настройки окружения.

Импорт и экспорт данных поддерживаются в формате JSON, обеспечивая совместимость с внешними системами. Для администраторов предусмотрена возможность массового импорта нескольких .docx файлов одновременно, а

также полного экспорта всей базы данных (студенты, отчёты, разделы, фрагменты и связи) для резервного копирования или миграции.

Целостность данных обеспечивается валидацией на клиенте и сервере (проверка форматов полей, обязательных атрибутов, корректности загружаемых файлов). Реализовано каскадное удаление связанных сущностей при удалении студента или отчёта, а также автоматическое удаление текстовых фрагментов, которые перестали использоваться ни в одном отчёте. База данных предварительно заполняется тестовыми данными для демонстрации всех возможностей системы.

Функциональные требования включают полный набор CRUD-операций для отчётов и студентов, поиск и фильтрацию ленты отчётов по группе, названию, автору, предмету, статусу, дате загрузки, индексу Флеша и оригинальности, а также полнотекстовый поиск по содержимому отчётов. Предусмотрены детальный просмотр отчёта с возможностью оставления комментария проверяющим, проверка на плагиат с визуальным сравнением двух отчётов, интерактивная визуализация графа связей, управление профилями студентов и генерация статистики с настраиваемыми группировками.

Производительность и масштабируемость обеспечены оптимизацией Cypher-запросов для типичных сценариев: фильтрация и пагинация ленты отчётов, выборка отчётов конкретного студента, поиск пересекающихся текстовых фрагментов между отчётами, подсчёт оригинальности и статистических метрик. В Neo4j созданы индексы по полям `id` для всех типов узлов и по полю `hash` для узлов `Chunk`, что значительно ускоряет поиск фрагментов при проверке на плагиат и импорте данных.

2. Сценарии использования



Главная страница (Дашборд)

Центральный узел приложения. Представляет собой таблицу со списком всех загруженных отчетов в системе. Основные элементы: таблица с колонками ID, Название, Автор, Группа, Статус, Дата изменения, а также кнопка быстрого поиска и кнопки перехода к массовым операциям импорта и экспорта. Функции страницы: просмотр общего состояния базы данных, отслеживание статуса обработки файлов в обработке или готов или ошибка. Из этой таблицы осуществляется переход на страницу конкретной сущности для детального просмотра отчета.

Поиск по фильтрации

Экран расширенного поиска для работы с большими объемами данных. Содержит отдельные поля ввода для каждого атрибута: название документа, автор, группа, содержит слово лемма, а также слайдер для выбора минимального индекса удобочитаемости. Особенностью страницы является реализация многокритериальной фильтрации, при этом поиск по текстовым полям выполняется регистронезависимо и осуществляется по подстроке.

Результат поиска по фильтрации

Вторичный экран списка, отображающий только те документы, которые соответствуют критериям из предыдущего экрана поиска. Представляет собой таблицу отфильтрованных данных. Функция страницы: позволяет быстро сузить выборку и перейти к анализу конкретного подходящего отчета.

Страница сущности (детальный просмотр отчета)

Страница для глубокой работы с одним конкретным документом. Содержит следующие элементы: метаданные автор, группа, статус, даты загрузки и редактирования, а также блок статистики с индексом Флеша, плотностью ключевых слов и наличием списка литературы. Важной особенностью является наличие редактируемого поля «Комментарий» для произвольных заметок пользователя и кнопки перехода к визуализации графа или анализу плагиата.

Граф отчета

Интерактивная визуализация внутренней структуры документа, построенная на основе данных из Neo4j. Элементы графа: узлы, представляющие документ, разделы, текстовые фрагменты, а также кнопки масштабирования и центрирования. Функция страницы: позволяет визуально оценить связность текста, иерархию разделов и выделить фрагменты, которые используются в нескольких отчётах.

Просмотр на плагиат

Интерфейс для детального сравнения текстов. Представляет собой сплит-вью с двумя окнами: слева отображается текст проверяемого отчета, справа — текст источника-донора. Ключевая функция: подсветка совпадающих фрагментов красным цветом. Сверху отображается итоговый процент заимствований и оригинальности.

Массовый импорт — загрузка

Экран для первичной загрузки данных в систему. Содержит область Drag-and-drop для загрузки zip-архивов или группы файлов в формате docx. Функция страницы: позволяет загружать целые наборы студенческих работ за один раз, что значительно ускоряет наполнение базы данных.

Массовый импорт — обработка

Экран мониторинга состояния загрузки. Элементы: индикатор прогресса в виде Progress Bar и список файлов с текущим статусом «Готов», «В обработке» или «Ошибка». Функция страницы: визуализация процесса парсинга и наполнения графовой базы данных в реальном времени с обратной связью по каждому файлу.

Экран статистики

Интерфейс для показа статистики по отчётности. Содержит фильтр по группе, чекбоксы выбора метрик для выгрузки, такие как объем отчётов, средняя оригинальность, средний индекс Флеша, среднее количество слов и другие. Функция страницы: формирование кастомизируемой статистики по выбранным параметрам для внешнего анализа и построение гистограмм на основе полученных данных.

3. Модель данных

Нереляционная модель

Графическое представление модели в графовой схеме



Узлы:

Student - студент, загружающий работы.

Report - загруженный отчет по лабораторной работе.

Part - структурная часть отчета (введение, заключение, основная часть и т.д.).

Chunk - фрагмент текста работы, используемый для поиска совпадений.

Связи:

[SUBMITTED] - от Student к Report.

[HAS_PART] - от Report к Part.

[CONTAINS] - от Part к Chunk.

Описание назначений коллекций, типов данных и сущностей

Student:

id: Целое число (Integer, 8 байт).

name: Строка (String, в среднем 16 байт).
surname: Строка (String, в среднем 16 байт).
group: Целое число (Integer, 8 байт).

Итого чистых данных: ~48 байт.

Report:

id: Целое число (Integer, 8 байт).
title: Строка (String, в среднем 34 байта).
author: Строка (String, в среднем 32 байта).
group: Целое число (Integer, 8 байт).
subject: Строка (String, в среднем 16 байт).
upload_date: Дата и время (DateTime/Integer, 8 байт).
words_count: Целое число (Integer, 8 байт).
flesh_index: Целое число (Integer, 8 байт).
keyword_density: Целое число (Integer, 8 байт).
originality: Число с плавающей точкой (Float, 8 байт).
conclusion: Логический тип (Boolean, 1 байт).
bibliography: Логический тип (Boolean, 1 байт).
introduction: Логический тип (Boolean, 1 байт).

Итого чистых данных: ~141 байт.

Part:

id: Целое число (Integer, 8 байт).
type: Строка (String, в среднем 10 байт).

Итого чистых данных: ~18 байт.

Chunk (Фрагмент текста):

id: Целое число (Integer, 8 байт).
text: Строка (String, в среднем 100 байт).
hash: Строка (String, 32 байта для SHA-256).

Итого чистых данных: ~140 байт.

Оценка объема информации, хранимой в модели

Введем переменную N - общее количество отчетов по лабораторным работам.

Для расчета примем следующие допущения:

В среднем один студент сдает 5 работ - количество студентов $0.2N$. Один отчет в среднем состоит из 3 частей (Part) - количество связей [:HAS_PART]: $3N$. Один отчет в среднем разбивается на 100 фрагментов (распределенных по частям) - общее количество связей [:CONTAINS]: $100N$. Пусть уникальных Chunk в базе будет $80N$ - количество связей [:SUBMITTED]: N .

Архитектура хранения Neo4j использует фиксированные размеры записей:

Узел (15 байт)

Связь (34 байта)

Блок свойств (41 байт).

Объекты графа:

Количество узлов: $(0.2 + 1 + 3 + 80)N = 84.2N$. Их объем: $84.2N * 15 = 1263N$ байт.

Количество связей: $(1 + 3 + 100)N = 104N$. Их объем: $104N * 34 = 3536N$ байт.

Исходя из размера чистых данных, допустим, что Student занимает 2 блока свойств, Report - 4 блока, Part - 1 блок, Chunk - 4 блока.

Общее количество блоков: $(0.2 * 2 + 1 * 4 + 3 * 1 + 80 * 4)N = 327.4N$.

Объем блоков свойств: $327.4N * 41 = 13423.4N$ байт.

Формула общего объема:

$$V(N) = 1263N + 3536N + 13423.4N = 18222.4N \text{ байт}$$

Избыточность данных

Посчитаем чистый объем данных $V_{raw}(N)$ без учета служебной информации СУБД:

Студенты: $0.2N * 48 = 9.6N$ байт.

Отчеты: $N * 141 = 141N$ байт.

Части отчета: $3N * 18 = 54N$ байт.

Фрагменты: $80N * 140 = 11200N$ байт.

$V_{raw}(N) = 9.6N + 141N + 54N + 11200N = 11404.6N$ байт

Формула зависимости избыточности от N :

$R_{neo4j}(N) = (V_{neo4j}(N)) / (V_{raw}(N)) = 18222.4N / 11404.6N = 1.60$

Избыточность является константой 1.60.

Направление роста модели

При увеличении количества объектов модель растет линейно ($O(N)$). Наибольший рост объема происходит за счет создания новых узлов `Chunk` (содержащих текст и хэш) и связей `[:CONTAINS]`. Если система начнет находить больше плагиата (количество переиспользуемых фрагментов возрастет), рост узлов `Chunk` замедлится, но рост связей останется строго линейным - по 100 связей на каждый новый отчет.

Примеры данных

Пример части отчета

ВВЕДЕНИЕ

Целью данной работы является создание программы для обработки PNG-изображений с использованием командной строки (CLI). Программа должна будет обеспечивать проверку соответствия файлов формату BMP.

Для достижения цели необходимо выполнить следующие задачи:

Изучение формата BMP.

Разработка CLI для взаимодействия с пользователем и обработки команд.

Реализация функций для обработки изображений, включая:

Замена параметров RGB у пикселя.

Обеспечение проверки BMP-формата и корректной обработки ошибок.

Реализация выравнивания данных в файле и сохранение стандартных значений BMP-заголовков.

Тестирование программы на различных входных данных.

Программа должна быть удобной в использовании, с четко определенными функциями и параметрами для обработки изображений. Все операции должны быть реализованы в виде отдельных функций, что облегчит тестирование и дальнейшее расширение функционала программы.

Пример данных (Узлы)

JSON

```
// Узел Student
{ "id": 1, "name": "Иван", "surname": "Иванов", "group": 3341 }

// Узел Report
{ "id": 10, "title": "Разработка CLI для обработки изображений",
"author": "Иванов И.И.", "group": 3341, "subject": "программирование",
"upload_date": 1672531200, "words_count": 1250, "flesh_index": 45,
"keyword_density": 3, "originality": 85.5, "conclusion": true, "bibliography": true,
"introduction": true }

// Узел Part
{ "id": 3, "type": "введение" }

// Узлы Chunk
{ "id": 6, "text": "Целью данной работы является создание программы
для обработки PNG-изображений с использованием командной строки (CLI)",
"hash":
"e6eef69985335996d94f37c4dbb5e4366c07d0d0e4fc283b3c3e1b27c39567d4" }
{ "id": 7, "text": "Программа должна будет обеспечивать проверку
соответствия файлов формату BMP.", "hash":
"5d3a15d5d274051f7bca2cb04b6a35a56b4c48770814df48dfbed0e534271b1d" }
{ "id": 8, "text": "Для достижения цели необходимо выполнить
следующие задачи: Изучение формата BMP.", "hash":
"e6537f76f4e74ec23f440fc9d2d47983a1473048e46d25fe05f22f27ccf52b8c" }
```

Пример данных (Связи)

JSON

// СВЯЗЬ SUBMITTED

(Student {id: 1}) -[:SUBMITTED]-> (Report {id: 10})

// СВЯЗЬ HAS_PART

(Report {id: 10}) -[:HAS_PART]-> (Part {id: 3})

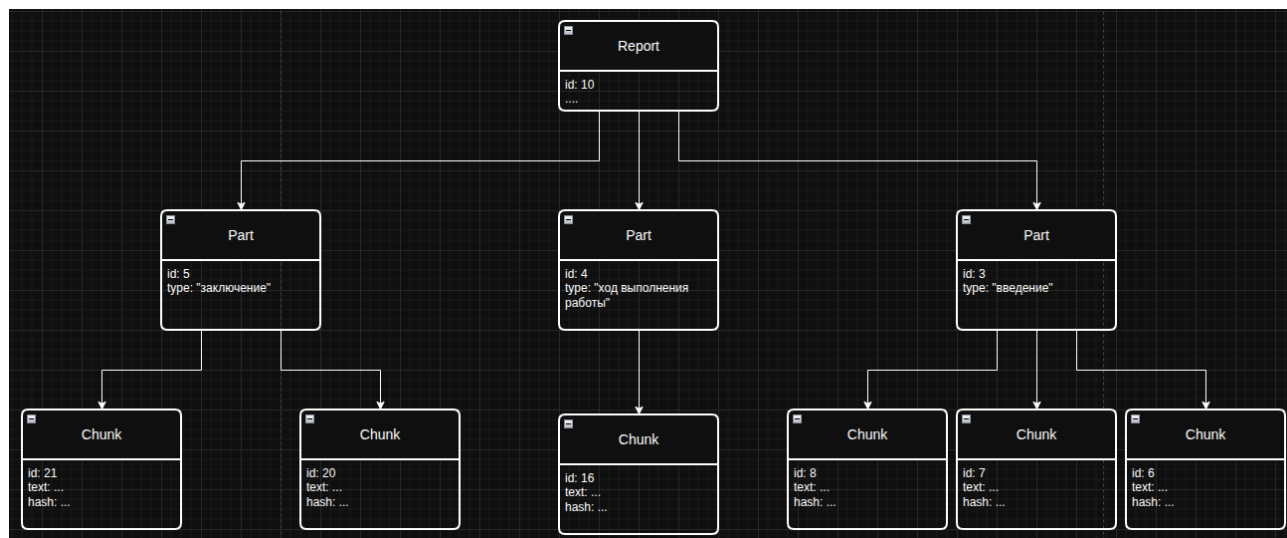
// СВЯЗЬ CONTAINS

(Part {id: 10}) -[:CONTAINS]-> (Chunk {id: 6})

(Part {id: 10}) -[:CONTAINS]-> (Chunk {id: 7})

(Part {id: 10}) -[:CONTAINS]-> (Chunk {id: 8})

графическое представление связей



Примеры запросов

Сценарий 1: Поиск всех работ конкретного студента

Cypher

```
MATCH (s:Student {id: 1})-[:SUBMITTED]->(r:Report)
```

```
RETURN r.title, r.upload_date, r.originality
```

Сценарий 2: Поиск плагиата

Cypher

```
MATCH (r1:Report {id: 101})-[:HAS_PART]->(:Part)-[:CONTAINS]->(c:Chunk)<-[:CONTAINS]-(:Part)<-[:HAS_PART]-(r2:Report)
```

```
WHERE r1 <> r2
```

```
RETURN r2.id AS SuspiciousReport, COUNT(c) AS SharedChunks
```

```
ORDER BY SharedChunks DESC
```

```
LIMIT 5
```

Сценарий 3: Поиск кол-во слов

```
MATCH (r:Report {id: 101})-[:HAS_PART]->(:Part)-[:CONTAINS]->(c:Chunk)
WITH r, sum(size(split(c.text, ' '))) AS total_words
SET r.words_count = total_words
RETURN r.words_count
```

Сценарий 4: Поиск индекса флеша

```
MATCH (r:Report {id: 101})-[:HAS_PART]->(:Part)-[:CONTAINS]->(c:Chunk)
WITH r, c.text AS txt
WITH r,
sum(size(split(txt, ' '))) AS words,
sum(size(split(txt, '.'))) AS sentences,
sum(size(txt)) / 3.0 AS estimated_syllables
SET r.flesh_index = toInteger(206.835 - 1.015 * (toFloat(words)/sentences) -
84.6 * (estimated_syllables/words))
RETURN r.flesh_index
```

Сценарий 5: Поиск плотности ключевых слов

```
MATCH (r:Report {id:
101})-[:HAS_PART]->(:Part)-[:CONTAINS]->(c:Chunk)
  WITH r, c.text AS txt
  WITH r,
  sum(size(split(txt, ' '))) AS total_words,
  sum(size(split(toLower(txt), keyword)) - 1) AS keyword_count
  SET r.keyword_density = toInteger((toFloat(keyword_count) / total_words)
* 100)
  RETURN r.keyword_density
```

Сценарий 6: Поиск процента оригинальности

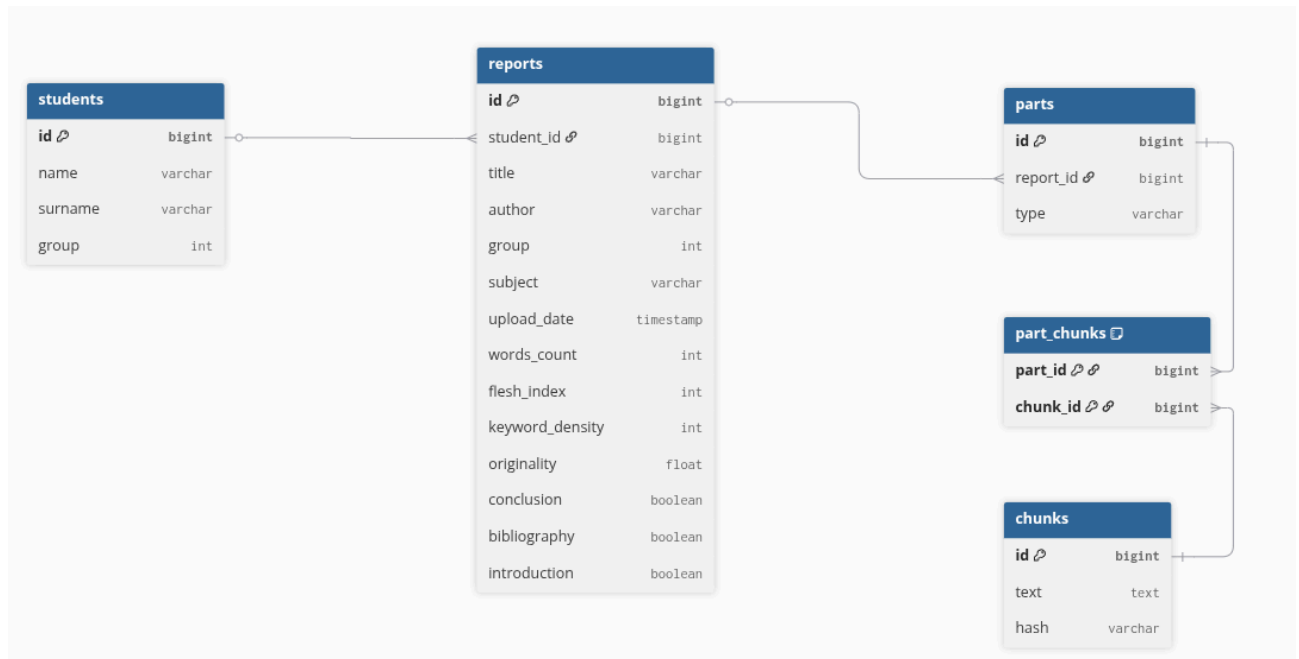
```
MATCH (r:Report {id:
101})-[:HAS_PART]->(:Part)-[:CONTAINS]->(c:Chunk)
  WITH r, c, count{ (c)<-[:CONTAINS]-(:Part)<-[:HAS_PART]-(:Report) }
  AS usage_count
  WITH r, count(c) AS total_chunks, sum(CASE WHEN usage_count = 1
THEN 1 ELSE 0 END) AS unique_chunks
  SET r.originality = (toFloat(unique_chunks) / total_chunks) * 100.0
  RETURN r.originality
```

Количество запросов для юзкейсов: 1 атомарный запрос на каждый из 6 сценариев.

Количество задействованных коллекций (типов узлов): В первом сценарии задействовано 2 (Student, Report), во всех остальных сценариях (со 2-го по 6-й) задействовано 3 (Report, Part, Chunk).

Реляционная модель

Графическое представление модели



Описание таблиц, типов данных и сущностей

Таблица Students

id: (BIGINT, 8 байт)
name: (VARCHAR, ~16 байт)
surname: (VARCHAR, ~16 байт)
group: (INT, 4 байта)

Таблица Reports

id: (BIGINT, 8 байт)
student_id: (BIGINT, 8 байт, Foreign Key)
title: (VARCHAR, ~34 байта)
author: (VARCHAR, ~32 байта)

group: (INT, 4 байта)
subject: (VARCHAR, ~16 байт)
upload_date: (TIMESTAMP, 8 байт)
words_count: (INT, 4 байта)
flesh_index: (INT, 4 байта)
keyword_density: (INT, 4 байта)
originality: (FLOAT, 8 байт)
conclusion: (BOOLEAN, 1 байт)
bibliography: (BOOLEAN, 1 байт)
introduction: (BOOLEAN, 1 байт)

Таблица Parts

id: (BIGINT, 8 байт)
report_id: (BIGINT, 8 байт, Foreign Key)
type: (VARCHAR, ~10 байт)

Таблица Chunks

id: (BIGINT, 8 байт)
text: (TEXT, ~100 байт)
hash: (CHAR(32), 32 байта)

Таблица Part_Chunks (Связующая кросс-таблица)

part_id: (BIGINT, 8 байт)
chunk_id: (BIGINT, 8 байт)

Оценка объема информации, хранимой в модели

Используем ту же переменную N (количество отчетов) и те же допущения. В PostgreSQL каждая строка имеет заголовок (Tuple Header) размером около 24 байт.

$$\text{Students: } 0.2N * (24 + 8 + 16 + 16 + 4) = 13.6N \text{ байт.}$$

$$\text{Reports: } N * (24 + 8 + 8 + 34 + 32 + 4 + 16 + 8 + 4 + 4 + 4 + 8 + 1 + 1 + 1) = 157N \text{ байт.}$$

$$\text{Parts: } 3N * (24 + 8 + 8 + 10) = 150N \text{ байт.}$$

$$\text{Chunks: } 80N * (24 + 8 + 100 + 32) = 13120N \text{ байт.}$$

$$\text{Part_Chunks: } 100N * (24 + 8 + 8) = 4000N \text{ байт.}$$

$$V_{\text{sql}}(N) = 13.6N + 157N + 150N + 13120N + 4000N = 17440.6N \text{ байт}$$

(Примечание: для обеспечения скорости поиска потребуются B-Tree индексы на все Primary и Foreign ключи, что на практике увеличит объем $V_{\text{sql}}(N)$ еще примерно в 1.5-2 раза).

Избыточность данных

Чистый объем данных $V_{\text{raw}}(N)$ остается прежним - 11404.6N байт.

Формула зависимости избыточности (без учета индексов):

$$R_{\text{sql}}(N) = 17440.6N / 11404.6N = 1.53$$

Направление роста модели

Рост также линейный ($O(N)$). Главный источник роста базы данных - таблица Chunks (за счет хранения самих фрагментов текста) и связующая таблица Part_Chunks, в которую добавляется по 100 записей на каждый новый отчет. При увеличении масштаба базы размер индексов кросс-таблицы станет основным потребителем памяти.

Примеры данных

Пример части отчета

ВВЕДЕНИЕ

Целью данной работы является создание программы для обработки PNG-изображений с использованием командной строки (CLI). Программа должна будет обеспечивать проверку соответствия файлов формату BMP.

Для достижения цели необходимо выполнить следующие задачи:

Изучение формата BMP.

Разработка CLI для взаимодействия с пользователем и обработки команд.

Реализация функций для обработки изображений, включая:

Замена параметров RGB у пикселя.

Обеспечение проверки BMP-формата и корректной обработки ошибок.

Реализация выравнивания данных в файле и сохранение стандартных значений BMP-заголовков.

Тестирование программы на различных входных данных.

Программа должна быть удобной в использовании, с четко определенными функциями и параметрами для обработки изображений. Все операции должны быть реализованы в виде отдельных функций, что облегчит тестирование и дальнейшее расширение функционала программы.

Таблица Reports

id	studen_id	title	author	group	subject	upload_time	...
10	1	Разработка CLI для обработки изображений	Иванов И.И.	3341	программирование	1672531200	...

Таблица Parts

id	report_id	type
3	10	введение

Таблица Chunks

id	text	hash
6	Целью данной Работы является создание Программы для обработки PNG-изображений с Использованием командной Строки (CLI).	A66b75c7e80 Efc551146cdb 322E436ec571 C384e86f0e0d F76a0fa44429a F6d1
7	Программа должна будет обеспечивать проверку соответствия файлов Формату BMP.	5d3a15d5d27 4051f7bca2cb 04b6a35a56b 4c48770814d F48dfbed0e5 34271b1d
8	Для достижения цели необходимо выполнить следующие задачи: Изучение формата BMP.	E6537f76f4e 74ec23f440f C9d2d47983 A1473048e46 D25fe05f22f2 7ccf52b8c
...	...	...

Таблица Part_Chunks

part_id	chunk_id
3	6
3	7
3	8
...	...

Примеры запросов

Сценарий 1: Поиск всех работ конкретного студента
SQL

```
SELECT * FROM Reports WHERE student_id = 1;
```

Сценарий 2: Поиск плагиата

SQL

```
SELECT p2.report_id AS SuspiciousReport, COUNT(*) AS SharedChunks
FROM Parts p1
JOIN Part_Chunks pc1 ON p1.id = pc1.part_id
JOIN Part_Chunks pc2 ON pc1.chunk_id = pc2.chunk_id
JOIN Parts p2 ON pc2.part_id = p2.id
WHERE p1.report_id = 101 AND p2.report_id != 101
GROUP BY p2.report_id
ORDER BY SharedChunks DESC
LIMIT 5;
```

Сценарий 3: Поиск кол-во слов

```
UPDATE Reports
SET words_count = (
SELECT COALESCE(SUM(array_length(regexp_split_to_array(c.text,
\s+'), 1)), 0)
FROM Parts p
JOIN Part_Chunks pc ON p.id = pc.part_id
JOIN Chunks c ON pc.chunk_id = c.id
WHERE p.report_id = Reports.id AND Reports.id = 101
)
WHERE id = 101;
```

Сценарий 4: Поиск индекса флеша

```
UPDATE Reports
SET flesh_index = (
SELECT (
206.835 - 1.015 * (SUM(array_length(regexp_split_to_array(c.text, '\s+'),
1))::FLOAT / NULLIF(SUM(char_length(c.text) - char_length(replace(c.text, '.', ''))),
0))
-- 84.6 * ((SUM(char_length(c.text))/3.0) /
NULLIF(SUM(array_length(regexp_split_to_array(c.text, '\s+'), 1)), 0))
)::INT
FROM Parts p
JOIN Part_Chunks pc ON p.id = pc.part_id
JOIN Chunks c ON pc.chunk_id = c.id
WHERE p.report_id = Reports.id AND Reports.id = 101
)
WHERE id = 101;
```

Сценарий 5: Поиск плотности ключевых слов

```
UPDATE Reports
SET keyword_density = (
SELECT (
SUM((char_length(c.text) - char_length(replace(lower(c.text), keyword, '')))
/ char_length('алгоритм'))::FLOAT
/ NULLIF(SUM(array_length(regexp_split_to_array(c.text, '\s+', 1)), 0) *
100
)::INT
FROM Parts p
JOIN Part_Chunks pc ON p.id = pc.part_id
JOIN Chunks c ON pc.chunk_id = c.id
WHERE p.report_id = Reports.id AND Reports.id = 101
)
WHERE id = 101;
```

Сценарий 6: Поиск процента оригинальности

```
WITH ChunkUsage AS (  
    SELECT c.id AS chunk_id, COUNT(DISTINCT p.report_id) AS  
usage_count  
    FROM Chunks c  
    JOIN Part_Chunks pc ON c.id = pc.chunk_id  
    JOIN Parts p ON pc.part_id = p.id  
    GROUP BY c.id  
),  
ReportStats AS (  
    SELECT p.report_id,  
    COUNT(c.id) AS total_chunks,  
    SUM(CASE WHEN cu.usage_count = 1 THEN 1 ELSE 0 END) AS  
unique_chunks  
    FROM Parts p  
    JOIN Part_Chunks pc ON p.id = pc.part_id  
    JOIN Chunks c ON pc.chunk_id = c.id  
    JOIN ChunkUsage cu ON c.id = cu.chunk_id  
    WHERE p.report_id = 101  
    GROUP BY p.report_id  
)  
UPDATE Reports  
SET originality = (rs.unique_chunks::FLOAT / NULLIF(rs.total_chunks, 0))  
* 100.0  
FROM ReportStats rs  
WHERE Reports.id = rs.report_id;
```

Количество запросов для юзкейсов: 1 запрос на каждый сценарий.

Количество задействованных коллекций (таблиц):

Для первого сценария — 1 таблица (Reports).

Для второго сценария — 2 таблицы: Parts и связующая таблица Part_Chunks, к которым применяются сложные ресурсоемкие операции Self-JOIN.

Для сценариев 3, 4, 5 и 6 — 4 таблицы: Reports, Parts, Part_Chunks и Chunks. Во всех этих случаях применяются множественные операции JOIN для сбора текстовых данных и агрегатные функции, что создает значительную вычислительную нагрузку на реляционную БД по сравнению с графовой.

Сравнение моделей

Удельный объем информации

Без учета индексов реляционная модель и графовая модель сопоставимы по размеру (~17440N против ~18222N байт). Однако Neo4j выигрывает в эффективном объеме за счет отсутствия необходимости создавать дополнительные junction-таблицы (как Part_Chunks).

Для эффективной работы SQL потребуются массивные индексы для связующей таблицы Part_Chunks (минимум два индекса: по part_id и по chunk_id) и внешних ключей в Parts. В Neo4j связи реализуются через "index-free adjacency" (безиндексную смежность) - каждая связь является прямым указателем в памяти. Таким образом, при прочих равных (для обеспечения сопоставимой скорости обхода связей) SQL-модель будет занимать значительно больший объем памяти.

Запросы по отдельным юзкейсам

Количество запросов: В обеих моделях логика реализуется за 1 атомарный запрос к СУБД.

Сложность выполнения:

SQL: Юзкейс поиска плагиата требует выполнения многократной операции JOIN: таблицы Parts с таблицей Part_Chunks (которая содержит сотни миллионов строк при масштабировании), затем снова с Part_Chunks и снова с Parts. Это приводит к экспоненциальному падению производительности при росте БД.

Neo4j: Обход графа происходит только по локальным связям тестируемого узла Report через промежуточные узлы Part. Сложность поиска зависит только от количества фрагментов в одном отчете и количества работ, использующих эти же фрагменты, а не от общего объема базы данных.

Вывод

Для реализации системы антиплагиата NoSQL(neo4j) модель является значительно более предпочтительной, чем классическая реляционная SQL(postgresql) модель. Так как в SQL это требует создания дополнительных сущностей (таблиц Parts) и тяжелых кросс-таблиц связи «многие-ко-многим» с выполнением дорогостоящих и множественных операций JOIN. NoSQL, благодаря своей архитектуре index-free adjacency, позволяет выполнять нативный обход связей между отчетами, их частями и текстовыми фрагментами в реальном времени, обеспечивая высокую производительность даже при миллионах загруженных документов.

4. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

4.1. Краткое описание

Приложение реализовано в виде трёхконтейнерной архитектуры, управляемой через Docker Compose. Первый контейнер db содержит графовую СУБД Neo4j 5.18 Community Edition, которая хранит все данные об отчётах, студентах, разделах и текстовых фрагментах, а также о связях между ними. Данные персистируются через Docker volume. Второй контейнер backend содержит серверную часть на Python 3.12 с фреймворком FastAPI. Он предоставляет REST API для всех операций: CRUD-операции над отчётами и студентами, поиск и фильтрация, проверка на плагиат, статистика, импорт и экспорт данных. Взаимодействие с Neo4j осуществляется через официальный Python-драйвер. Третий контейнер frontend содержит клиентскую часть, реализованную с использованием серверного рендеринга на Jinja2 с шаблонами Bootstrap 5. Клиентская часть реализует все необходимые интерфейсы: ленту отчётов, детальный просмотр, формы добавления и редактирования, страницы поиска, статистики и визуализации графов.

4.2. Используемые технологии

Для реализации серверной части (backend) использован язык Python версии 3.12 с веб-фреймворком FastAPI 0.111 и сервером Uvicorn 0.29. Взаимодействие с графовой базой данных осуществляется через официальный драйвер neo4j 5.19.0. Для обработки загружаемых документов в формате .docx применяется библиотека python-docx 1.1.0, которая позволяет извлекать текст, распознавать заголовки разделов и разбивать содержимое на структурные элементы. Шаблонизация HTML-страниц выполняется с помощью Jinja2 3.1.4. Для аутентификации пользователей используется библиотека itsdangerous 2.2.0, обеспечивающая создание защищённых сессионных cookie. Работа с multipart-формами при загрузке файлов реализована через python-multipart 0.0.9, а асинхронная работа с файлами — через aiofiles 23.2.1. Валидация данных на уровне приложения выполняется с помощью Pydantic 2.7.1.

Клиентская часть (frontend) реализована с использованием серверного рендеринга на базе шаблонизатора Jinja2, что позволяет динамически формировать

HTML-страницы на стороне сервера. Для стилизации интерфейса применяется Bootstrap 5.3.3, обеспечивающий адаптивный дизайн и готовые компоненты интерфейса, а также Bootstrap Icons 1.11.3 для иконографии. Визуализация графа связей отчёта выполнена с помощью библиотеки Cytoscape.js 3.28.1, которая позволяет интерактивно масштабировать, перетаскивать и исследовать граф, состоящий из узлов-документов, узлов-разделов и узлов-текстовых фрагментов. Построение статистических диаграмм реализовано на Chart.js 4.4.1, который отображает гистограммы по выбранным пользователем группировкам и метрикам.

В качестве системы управления базами данных выбрана Neo4j 5.18 Community Edition — графовая СУБД, которая хранит все сущности (отчёты, студенты, разделы, текстовые фрагменты) в виде узлов, а связи между ними (SUBMITTED, HAS_PART, CONTAINS) — в виде рёбер.

Вся инфраструктура приложения контейнеризирована с помощью Docker и оркестрируется через Docker Compose, что позволяет развернуть систему одной командой без ручной установки зависимостей и настройки окружения.

ЗАКЛЮЧЕНИЕ

Реализовано веб-приложение для проверки студенческих отчётов на плагиат с использованием графовой базы данных Neo4j. Приложение позволяет загружать документы в формате .docx, автоматически извлекать из них текст, разбивать на структурные разделы (введение, основная часть, заключение, список литературы) и текстовые фрагменты с вычислением криптографических хешей для каждого фрагмента. На основе анализа пересекающихся фрагментов между отчётами система вычисляет процент оригинальности каждого документа, выявляет источники заимствований и визуализирует связи между отчётами в виде интерактивного графа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Neo4j Documentation [Электронный ресурс]. - Режим доступа: <https://neo4j.com/docs/> .
2. FastAPI Documentation [Электронный ресурс]. - Режим доступа: <https://fastapi.tiangolo.com/> .
3. Docker Documentation [Электронный ресурс]. - Режим доступа: <https://docs.docker.com/> .
4. Cypher Query Language [Электронный ресурс]. - Режим доступа: <https://neo4j.com/docs/cypher-manual/current/> .
5. Jinja2 Documentation [Электронный ресурс]. - Режим доступа: <https://jinja.palletsprojects.com/> .
6. Bootstrap Documentation [Электронный ресурс]. - Режим доступа: <https://getbootstrap.com/docs/5.3/> .
7. Cytoscape.js Documentation [Электронный ресурс]. - Режим доступа: <https://js.cytoscape.org/> .
8. Chart.js Documentation [Электронный ресурс]. - Режим доступа: <https://www.chartjs.org/docs/latest/> .
9. python-docx Documentation [Электронный ресурс]. - Режим доступа: <https://python-docx.readthedocs.io/> .

ПРИЛОЖЕНИЕ А

Документация по сборке и развертыванию

1) клонировать репозиторий:

```
git clone ssh-of-repo
```

2) запустить docker compose

```
sudo docker compose up --build
```

доступны тестовые пользователи:

admin / admin123

teacher / teacher123